

Dual System VLA on LIBERO-Long: Controlled Ablation

Soroosh Dini-Mortezapoor

1 Introduction

This compact report summarizes a dual-system VLA evaluated on the ten-task LIBERO-Long `libero_10` suite.¹ System 2 selects a task phase, which is mapped to a shared skill identity and embedded as extra conditioning input to System 1 (an Action Chunking Transformer trained to associate skills with action chunks). A seven-condition ablation is designed to evaluate this system against a single VLA system.

2 Problem Statement

The evaluation addresses three question groups.

1. Counterfactual: (Q1.1) Does the conditioning channel matter at all, or is System 1 ignoring it? What happens to the success rate if System 2 is **(Q1.2)** removed? **(Q1.3)** frozen (time-based open loop)? **(Q1.4)** replaced with a naive baseline (e.g., a static embedding of the first task skill)?

2. Error Recovery: (Q2.1) Does System 2 actually help System 1 recover from out-of-distribution perturbations or compounding errors? When the selector detects a disturbance, does it **(Q2.2)** revise the progress estimate backward? **(Q2.3)** improve success?

3. Bottleneck Analysis: (Q3.1) Is the primary bottleneck in the final system a failure of reasoning (System 2 predicting poor progress) or a failure of execution (System 1 failing to reach the sub-goal)? **(Q3.2)** How do we know?

3 System Design

The system consists of two components: System 1 (fast, reactive controller) and System 2 (slow, semantic progress estimator). They communicate through a single discrete integer interface that is recomputed every control step.

System 2 is a small neural network that estimates task progress from visual observations, proprioception, and task identity. It receives three inputs: (1) pooled visual features from ACT’s backbone (512-dimensional), (2) robot proprioception including end-effector position, orientation, and gripper state (8-dimensional), and (3) a task identity embedding (32-dimensional). The architecture consists of a two-layer MLP for processing proprioception (64 hidden units), concatenated with visual features and task embedding, followed by a classification head (256 hidden units) that outputs logits over possible phase values.

System 2 contains approximately 2M parameters and runs at 10 Hz, matching the control frequency of the LIBERO simulation environment. It outputs a phase value in the range $[0, n_{\text{phases}} - 1]$, where n_{phases} varies by task (2–4 phases per task). The selector masks invalid phases for each task before argmax, ensuring only valid phases are selected. When System 2 makes an erroneous prediction (e.g., due to visual ambiguity or perturbation), it can output

a lower phase than the previous step, enabling backward transitions that represent error recovery behavior.

System 1 is a 51.6M-parameter Action Chunking Transformer (ACT) that predicts 100 future seven-dimensional joint actions from visual and proprioceptive inputs. ACT receives third-person camera images (256×256 RGB), wrist camera images, and the 8-dimensional proprioceptive state. The skill derived from System 2’s phase is embedded (64-dimensional) and projected to ACT’s model dimension (512), then added to ACT’s first encoder latent token through a plugin wrapper around the `encoder_latent_input_proj` layer.

ACT predicts action chunks of 100 steps but *executes only 10* actions before requesting a new prediction, yielding approximately 52 decisions per 520-step episode. This design maintains closed-loop control: if System 2 changes its phase prediction mid-chunk, the stale action queue is flushed immediately and a new chunk is computed from the current observation. Without this mechanism, at LIBERO’s 10 Hz control rate, executing all 100 predicted actions would commit to 10 seconds of motion per decision, effectively disabling the closed loop.

Shared skill identities (26 in total) are used as the interface between the two systems. Subgoals with identical semantic meaning across tasks share the same skill ID. For example, "grasp the white mug" uses the same skill ID in both task 4 and task 6, allowing ACT (System 1) to generalize across tasks while System 2 disambiguates through the task identity it receives explicitly. The index maps (task_id, phase) pairs to one of 25 skill IDs, plus a null ID (25) for "no subgoal given". System 2 outputs phase in per-task space, and the policy composes this with task identity (extracted from instruction text) to produce the global skill row that ACT embeds. This composition enables the ablation conditions: at evaluation time, the phase source can be swapped without retraining System 1.

Phase oscillation occurs when visual observations are ambiguous (e.g., an arm poised over an open drawer could indicate either "place" or "close drawer" phase). The selector, being memoryless by design, may flip between these interpretations across consecutive steps. Optional hysteresis (asymmetric thresholds: commit after 3 consistent forward predictions or 8 backward) can smooth these oscillations at inference time without affecting training.

Perturbation conditions (i.e. forced object drops, action noise, visual shifts), make System 2’s ability to predict backward phase transitions critical. A drop event causes the selector to output an earlier phase corresponding to re-grasping.

4 Implementation

The policy is implemented as an installable out-of-tree plugin `lerobot_policy_subgoal_act` for the LeRobot framework. This plugin architecture avoids forking the upstream

¹An extended version with detailed design and implementation choices, evaluation methodology, and in-depth discussion is available in the repository [here](#).

ACT implementation while enabling the subgoal conditioning mechanism.

4.1 Plugin Architecture

The main design goal was to add subgoal conditioning to ACT without copying the upstream LeRobot implementation, so I wrapped the smallest independent component: the `encoder_latent_input_proj` layer that builds ACT’s first encoder token. A custom `SubgoalEmbedding` holds a small embedding table (26 rows: 25 skill identities plus a null token) and a linear projection; it lets the original layer produce its token normally, then adds the projected subgoal embedding to it. This addition-based conditioning keeps ACT’s token structure intact and is simpler than concatenation or FiLM.

The policy class `SubgoalACTPolicy` inherits LeRobot’s `ACTPolicy` and overrides two methods: `select_action` injects the subgoal at inference (computed by System 2 or a predefined mode), and `forward` applies teacher forcing with ground-truth subgoals during training. The plugin registers with LeRobot’s policy registry, and an `eval_mode` configuration switch controls the subgoal source at evaluation time, enabling all ablation conditions from a single trained checkpoint.

5 Evaluation

5.1 Methodology

I train on 379 `libero_10` episodes from a pinned dataset revision and evaluate on ten LIBERO-Long tasks. Episodes have a 520-step limit. Evaluation uses paired comparisons: all conditions (variations) receive identical stored initial states in identical order across seeds 1000, 2000, and 3000. The main evaluator, therefore, uses 300 episodes per condition (100 per seed). I report pooled success rates with 95% Wilson confidence intervals and use paired McNemar tests on discordant outcomes. The three seeds change which episodes are evaluated, not the trained weights. No condition received separate hyperparameter tuning or best-of evaluation.

5.2 Ablation Conditions (Variations)

The ablation study comprises seven conditions across three trained checkpoints. Each condition isolates a different aspect of the two-system architecture and maps to specific questions from Section 2.

1. **A_learned** (reference): Uses the live System 2 selector to predict phase from observations. This is the full system as trained, serving as the primary reference point for all comparisons.
2. **A_debounced** (stability variant): Similar to **A_learned** with the same weights, and only applies hysteresis to the learned selector: commits to a new phase only after 3 consistent forward predictions or 8 backward predictions.
3. **C_none** (floor baseline): Removes System 2 entirely; System 1 receives no skill signal, as the subgoal embedding is set to the null row index. This is plain ACT without subgoal conditioning, trained separately to ensure fair comparison. Directly answers **Q1.1** (does the channel matter?) and **Q1.2**: if **C_none** performs similarly to **A_learned**, System 1 ignores the channel.

4. **B_static** (naive baseline): Receives the first task skill (i.e. the skill for phase 0) throughout the episode, implementing the constant subgoal baseline. Trained separately rather than simulated at evaluation time to avoid out-of-distribution behavior. Comparing **B_static** against **A_learned** answers **Q1.4**: does a dynamic signal beat a static one?
5. **A_frozen** (clock baseline): Receives a skill set by a fixed open-loop schedule based on elapsed steps: $\text{phase} = (\text{elapsed_steps} / \text{max_steps}) \times \text{n_phases}$, mapped to the corresponding skill. Cannot go backwards by construction. Comparing **A_frozen** against **A_learned** answers **Q1.3**: does closed-loop re-evaluation help, or is open-loop timing sufficient?
6. **A_shuffled**: Receives a random valid skill at each step, deliberately wrong. Expected to decrease performance if System 1 actually uses the subgoal channel. Answers **Q1.1**: if **A_shuffled** performs like **A_learned**, the channel is ignored and the architecture is broken.
7. **A_oracle** (upper bound): Receives the ground-truth skill from simulator state: BDDL goal predicates plus gripper proprioception. Cannot run through the standard evaluator, so measured in a custom matched rollout harness. Answers **Q3.1–Q3.2**: if **A_oracle** substantially outperforms **A_learned**, the bottleneck is System 2’s reasoning; if they are similar, the bottleneck is System 1’s execution capability.

Error recovery (Q2.1–Q2.3) is measured with a separate perturbation study rather than the seven-condition ablation: trained checkpoints are rolled out under **forced drops**, **action noise**, **object shifts**, and **visual shifts**, and forward/backward phase transitions are compared against the unperturbed baseline. This separates whether System 2 detects a disturbance (Q2.2) from whether the revision restores success (Q2.3).

5.3 Training Pipeline

Training proceeds in two stages: dataset labeling followed by joint training of both systems. LIBERO-Long tasks combine two to four stages and run for up to 520 steps at 10 Hz.

Stage 1: Label Generation. Ground-truth phase labels are derived from 379 LIBERO-10 demonstration episodes. Labels combine BDDL goal predicates with hysteretic gripper state transitions. BDDL predicates alone are insufficient because they omit the "currently grasped" state needed to distinguish grasp from place phases. The labeling script replays each episode, tracking gripper open/close transitions with dual thresholds (open above 0.65, closed below 0.35 of maximum gripper opening) to assign phase boundaries. Labels are stored as a Parquet file with frame-level (phase, task_id) pairs, indexed by dataset frame number.

Stage 2: Joint Training. Three conditions are trained separately: **A_learned** (both systems), **B_static** (System 1 with constant first-phase embedding), and **C_none** (System 1 without any subgoal conditioning). Training uses 100,000 steps, batch size 64, on the pinned LIBERO dataset revision. System 1 receives ground-truth subgoals (converted from the phase labels) during training (teacher forcing), preventing early System 2 errors from corrupting ACT’s learning. System 2 is trained alongside System 1 in **A_learned** on the ground-truth phase labels with a cross-entropy loss

on phase prediction, weighted at 0.1 relative to ACT’s action prediction loss. This separation allows attribution of failures at evaluation time: if `A_learned` underperforms `A_oracle`, the gap reflects System 2’s prediction quality rather than System 1’s capability.

The training data is filtered to include only the 10 LIBERO-10 tasks used in evaluation (379 episodes out of 1,693 total in the dataset). Training on all tasks degrades performance, particularly for unconditioned baselines that cannot distinguish tasks from text input.

5.4 Labeling and Data Flow

Ground-truth phase labels are stored as a sidecar Parquet file rather than modifying the dataset’s normalization metadata. The labeling script derives labels for 379 episodes (101,469 frames) from BDDL predicates and gripper state; segment counts disagree with intended subgoal boundaries in 24% of episodes, a label-quality limitation the system must tolerate. During training the policy loads labels lazily and applies subgoal dropout so the null row receives gradients; at evaluation the subgoal source is set by `eval_mode`, enabling every condition from three trained checkpoints.

5.5 Measurement

The standard evaluator cannot expose BDDL predicates, so `A_oracle` is measured with a custom matched rollout harness, and all comparisons are made within one measurement path. The harness logs full phase trajectories, forward/backward transition counts, failure phase, and oracle agreement, separating detection (Q2.1–Q2.2) from recovery outcome (Q2.3).

5.6 Ablation results

Table 1 reports the outcomes of this evaluation run: pooled success over 300 paired episodes per condition, with 95% Wilson intervals and the exact paired McNemar p relative to the `A_learned` reference.

Table 1: Pooled success over 300 paired episodes; 95% Wilson intervals. MCNEMAR is the exact paired two-sided test against the `A_learned` reference.

Condition	Success	MCNEMAR p
<code>A_learned</code>	50.7% [45.0, 56.3]	—
<code>A_shuffled</code>	34.0% [28.9, 39.5]	$< 10^{-7}$
<code>A_frozen</code>	50.0% [44.4, 55.6]	0.89
<code>B_static</code>	44.0% [38.5, 49.7]	0.06
<code>A_debounced</code>	50.3% [44.7, 56.0]	1.00
<code>C_none</code>	46.3% [40.8, 52.0]	0.27

5.7 Error recovery

The perturbation study measures recovery (Q2.1–Q2.3) by rolling `A_learned`, `A_frozen`, and `A_debounced` out under four interventions. Table 2 reports success and mean backward subgoal transitions per episode; `A_frozen` cannot go backwards by construction.

Table 2: Error recovery under perturbations (50 episodes per cell). BWD is mean backward subgoal transitions per episode; `A_frozen` cannot go backwards by construction.

Perturbation	<code>A_learned</code>		<code>A_frozen</code>		<code>A_debounced</code>	
	Success	BWD	Success	BWD	Success	BWD
None	52%	2.14	46%	0.00	56%	0.32
Forced drop	38%	2.80	26%	0.00	28%	1.40
Action noise	32%	1.68	34%	0.00	26%	0.64
Object shift	24%	2.80	20%	0.00	24%	0.92
Visual shift	54%	1.02	54%	0.00	44%	0.36

5.8 Bottleneck analysis

Q3.1–Q3.2 compare `A_oracle` against `A_learned` on the matched rollout harness (100 episodes per condition), since `A_oracle` cannot run through the standard evaluator. `A_learned` and `A_oracle` both succeed in 47% of episodes on this harness: giving System 2 perfect progress knowledge produces no improvement over the learned selector.

6 Discussion and Outlook

Q1.1: the channel is live. A random valid skill reduces success by 16.7 points (34.0% against 50.7%, $p < 10^{-7}$). System 1 therefore reads the skill token, and the paired protocol resolves this effect on 300 episodes, so the null results below come from a sensitive instrument.

Q1.2: removing System 2 costs 4.4 points (46.3% against 50.7%, $p = 0.27$), a difference this sample size cannot resolve. The asymmetry is the finding: a wrong skill token degrades success roughly four times as much as a correct one improves it. ACT already infers the stage from the images, so a correct token is largely redundant, while a contradictory one overrides the camera.

Q1.3: freezing System 2 into an open-loop clock changes nothing (50.0% against 50.7%, $p = 0.89$), and hysteresis changes nothing either (50.3%, $p = 1.00$). LIBERO-Long demonstrations share layouts and run to similar durations, so a schedule spread over the horizon is approximately correct most of the time. This bounds the value of online reasoning on this benchmark, not in general.

Q1.4: the naive baseline is worse than nothing. A constant first-skill embedding reaches 44.0%, below `C_none` at 46.3% and 6.7 points below `A_learned` ($p = 0.06$), since it is stale after the first grasp. Part of the `A_learned` over `B_static` gap therefore reflects avoiding harm rather than added conditioning value. Step counts show no separation either: successful episodes on the unperturbed control take 244 steps for `A_learned`, 255 for `A_frozen` and 267 for `A_debounced`. Conditioning alters which episodes succeed, not their duration.

Q2: System 2 notices, System 1 cannot act. A forced drop raises `A_learned`’s backward transitions from 2.14 to 2.80 per episode while `A_frozen` remains structurally at zero, so detection (Q2.1, Q2.2) works. The outcome (Q2.3) is suggestive only: `A_learned` leads `A_frozen` 38% to 26% under forced drop (paired $p = 0.07$), the class the mechanism targets, but pooled over all four perturbations the gap falls to 37% against 33.5% ($p = 0.30$). Among perturbed `A_learned` episodes, those with a backward transition succeed 13 of 97 against 61 of 103 without: a backward transition indicates a failure but does not reverse it.

Q3: the bottleneck is execution. `A_oracle` and `A_learned` both reach 47%, yet the selector agrees with the oracle only about 63% of the time, so real reasoning headroom existed and closing all of it produced no improvement. Three independent manipulations of System 2 (freeze, debounce, replace with ground truth) are null while corrupting the channel is decisive, which is consistent with an execution-bound system (Q3.2).

Outlook. The next effort should target System 1: a stronger backbone or a diffusion action head. A better reasoner cannot help until the controller can execute a subgoal it has already been given. Three training seeds would also make the four-point gaps interpretable.

Acknowledgment

AI coding assistants were used to support parts of this project: setting up the libraries and environment, accelerating documentation, and writing scripts for automating training, evaluation, and rollout-video generation on a distributed setup. The plugin and the dual-system architecture, their implementation, the design decisions about the interface, the analysis of the evaluations, and the writing of this report were carried out by me.